



Fuente: Arduino

Entorno de desarrollo Arduino

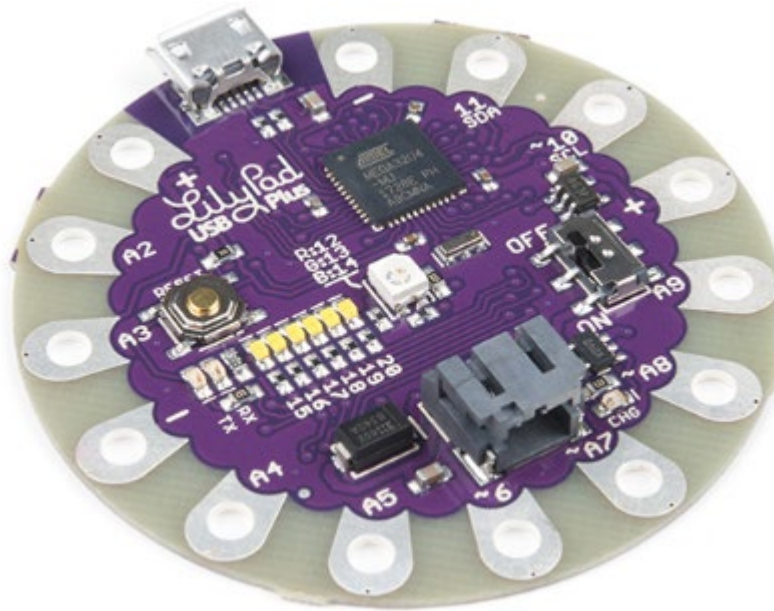
Dpto. de Electrónica Física, Ingeniería Eléctrica y
Física Aplicada

Juan Manuel López Navarro



LILYPAD UN TOOLKIT PARA WEARABLES

- Tarjeta desarrollada alrededor de un microprocesador Atmega 32U4.
- Especialmente desarrollada para prototipos wearables en textiles



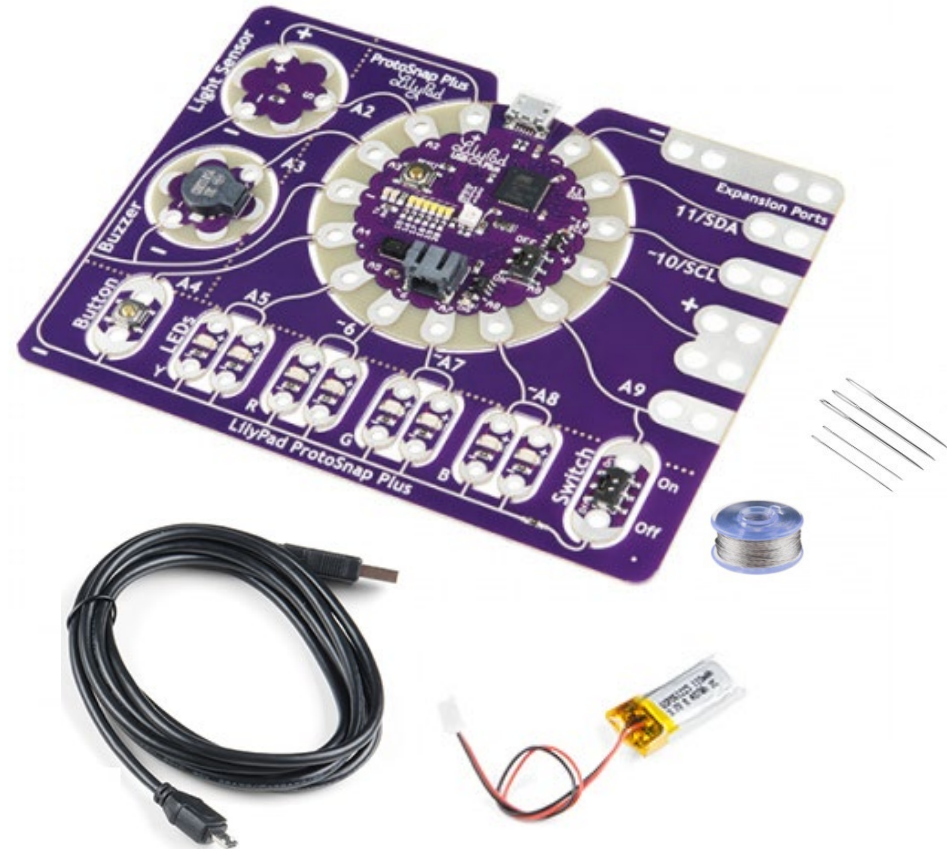
Fuente: Sparkfun Electronics



Kit de desarrollo

Periféricos

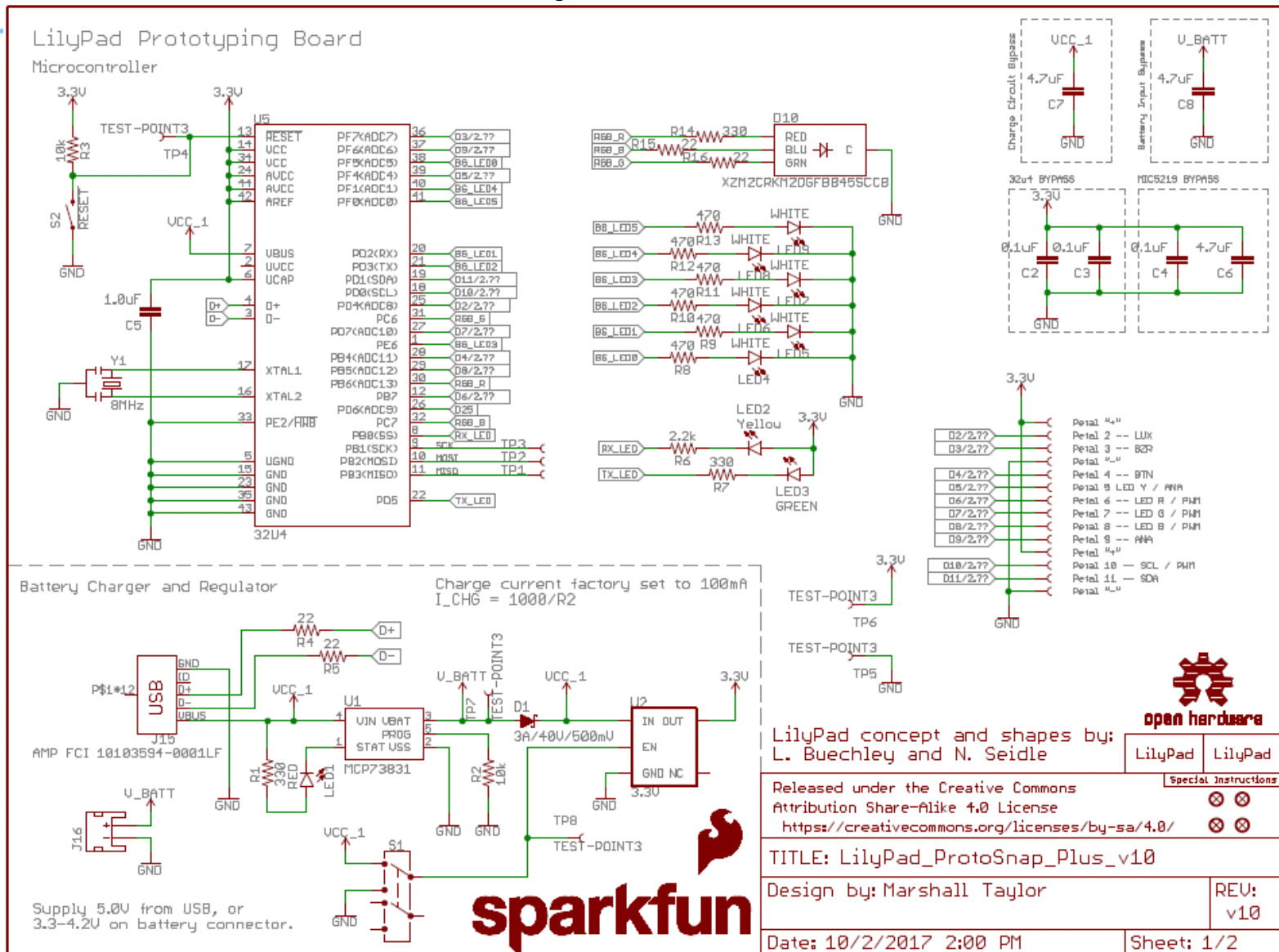
- ❑ Un pulsador
- ❑ Un interruptor
- ❑ Un LED RGB
- ❑ Un sensor de luz
- ❑ Una barra de leds blancos(6)
- ❑ Un zumbador
- ❑ Cuatro parejas de leds. (Rojo, Verde, Azul y Amarillo)



Fuente: Sparkfun Electronics

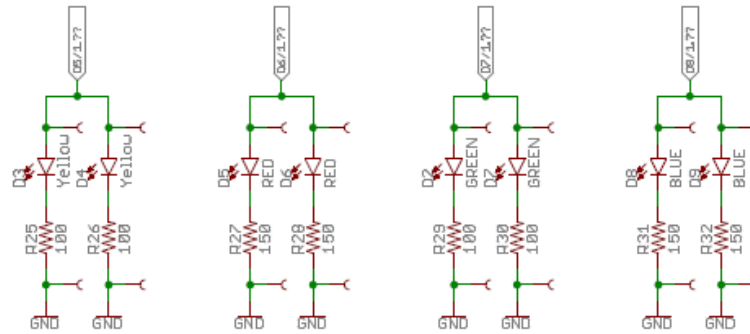


Esquemáticos



Esquemáticos

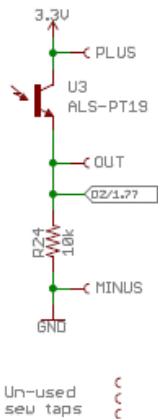
LEDS



Expansion tabs



Light Sensor



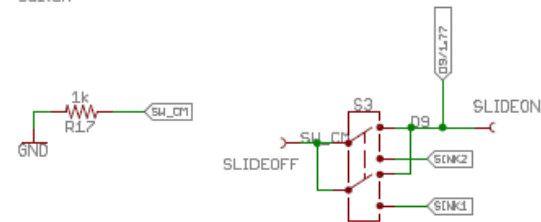
Buzzer



Button



Switch



Released under the Creative Commons
Attribution Share-Alike 4.0 License
<https://creativecommons.org/licenses/by-sa/4.0/>

TITLE: LilyPad_ProtoSnap_Plus_v10

Design by: Marshall Taylor

REV:
v10

Date: 10/2/2017 2:00 PM

Sheet: 2/2

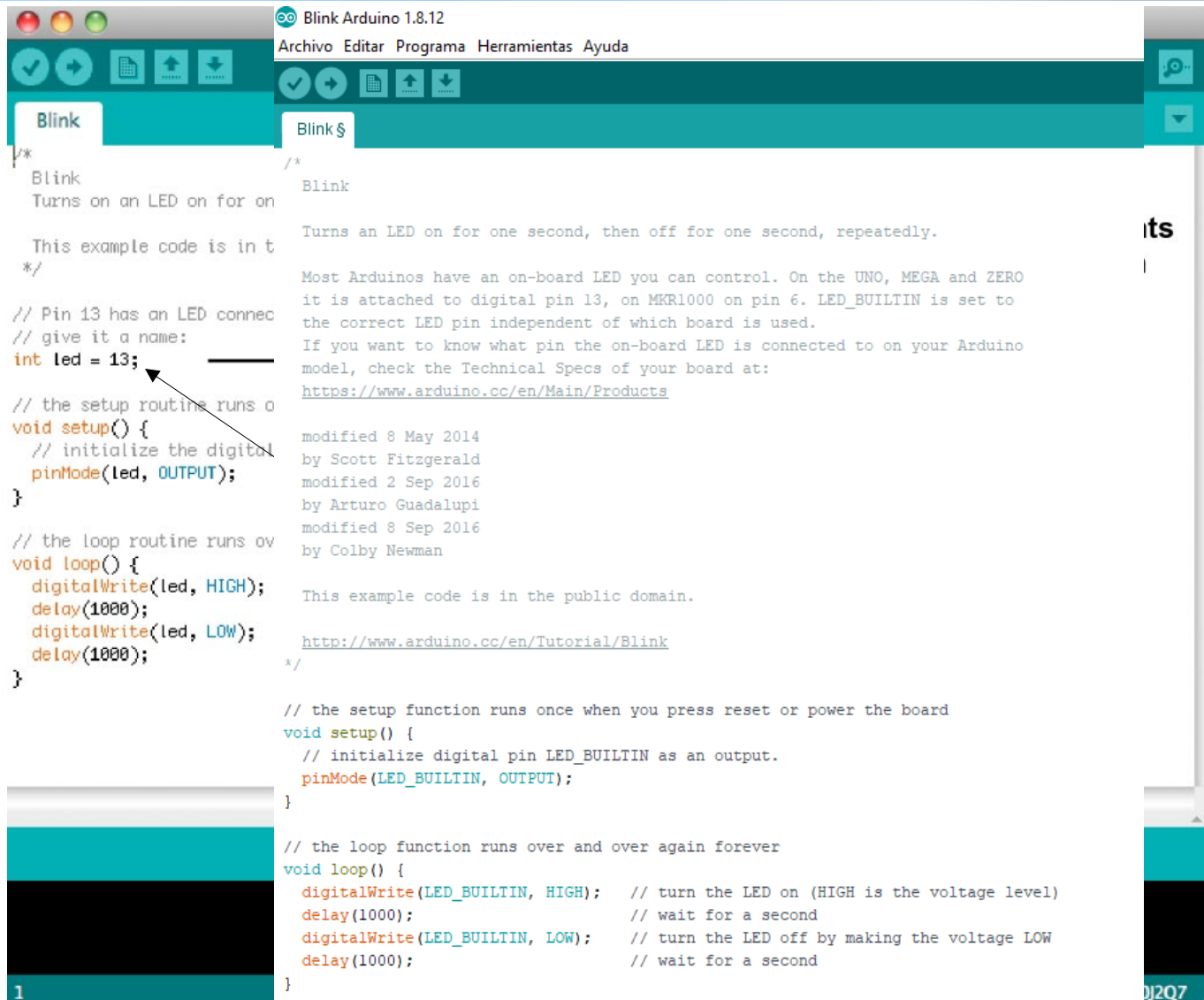


Arduino un sistema de desarrollo fácil y simple

- ❑ Arduino es una herramienta para el desarrollo de prototipos electrónicos basados en sistemas microprocesador .
- ❑ Se ha popularizado enormemente debido a su sencillez de uso, y su utilización por una enorme comunidad de usuarios que hacen diseños “amateur” que se publican en la web.
- ❑ Se dispone de un gran número de sensores y dispositivos de bajo coste, que abaratan los diseños y se encuentran ejemplos en casi cualquier campo donde se utiliza Arduino



Entorno



```
/*
 * Blink
 * Turns on an LED on for on
 *
 * This example code is in the
 */

// Pin 13 has an LED connected
// give it a name:
int led = 13;

// the setup routine runs once
void setup() {
  // initialize the digital
  pinMode(led, OUTPUT);
}

// the loop routine runs over
void loop() {
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
  delay(1000);
}

/*
 * Blink
 * Turns an LED on for one second, then off for one second, repeatedly.
 *
 * Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
 * it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
 * the correct LED pin independent of which board is used.
 * If you want to know what pin the on-board LED is connected to on your Arduino
 * model, check the Technical Specs of your board at:
 * https://www.arduino.cc/en/Main/Products
 *
 * modified 8 May 2014
 * by Scott Fitzgerald
 * modified 2 Sep 2016
 * by Arturo Guadalupi
 * modified 8 Sep 2016
 * by Colby Newman
 *
 * This example code is in the public domain.
 *
 * http://www.arduino.cc/en/Tutorial/Blink
 */

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000); // wait for a second
  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
  delay(1000); // wait for a second
}
```

Fuente: Arduino



Programas en Arduino (Sketch)

- Los programas en Arduino se denominan Sketch.
- Son lanzados desde el main, el cual no es accesible por el programador.

```
#include <Arduino.h>

// Declared weak in Arduino.h to allow user redefinitions.
int atexit(void (* /*func*/ )()) { return 0; }

// weak empty variant initialization function.
// May be redefined by variant files.
void initVariant() __attribute__((weak));
void initVariant() { }

void setupUSB() __attribute__((weak));
void setupUSB() { }

int main(void)
{
    init();

    initVariant();

    #if defined(USBCON)
        USBDevice.attach();
    #endif

    setup();

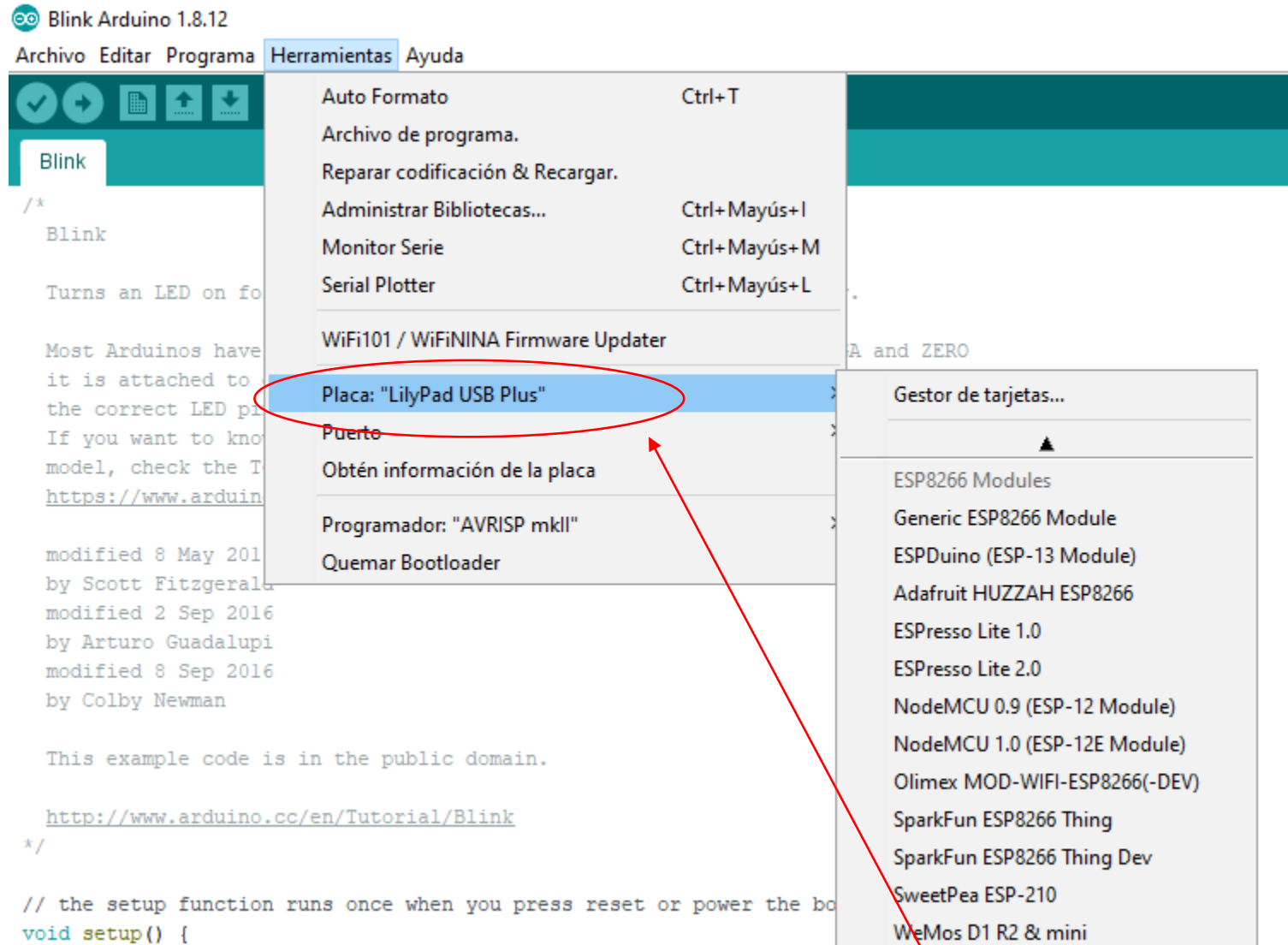
    for (;;) {
        loop();
        if (serialEventRun) serialEventRun();
    }

    return 0;
}
```

Fuente: Arduino



Selección del hardware



Fuente: Arduino

Hardware para internet de las cosas (IoT)



Selección del hardware

Blink Arduino 1.8.12

Archivo Editar Programa Herramientas Ayuda

Nuevo Ctrl+N
Abrir... Ctrl+O
Abrir Reciente
Proyecto
Ejemplos
Cerrar Ctrl+W
Salvar Ctrl+S
Guardar Como... Ctrl+Mayús+S
Configurar Página Ctrl+Mayús+P
Imprimir Ctrl+P
Preferencias Ctrl+Coma
Salir Ctrl+Q

modified 2 Sep 2016
by Arturo Guadalupi
modified 8 Sep 2016
by Colby Newman

This example code is in the pub.

<http://www.arduino.cc/en/Tutori>

```
// the setup function runs once w
void setup() {
  // initialize digital pin LED_B
  pinMode(LED_BUILTIN, OUTPUT);
}
```

```
// the loop function runs over an
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

Subido

El Sketch usa 3938 bytes (13%) del
Las variables Globales usan 149 by

Preferencias

Ajustes Red

Localización de proyecto
C:\Users\Administrador\Documents\Arduino Explorar

Editor de idioma: System Default (requiere reiniciar Arduino)

Editor de Tamaño de Fuente: 12

Escala Interfaz: ☒ Automático 100 % (requiere reiniciar Arduino)

Tema: Tema por defecto (requiere reiniciar Arduino)

Mostrar salida detallada mientras: ☐ Compilación ☐ Subir

Advertencias del compilador: Ninguno

☐ Mostrar números de línea ☐ Habilitar Plegado Código

☒ Verificar código después de subir ☐ Usar editor externo

☒ Comprobar actualizaciones al iniciar ☒ Guardar cuando se verifique o cargue

☐ Use accessibility features

Gestor de URLs Adicionales de Tarjetas: usercontent.com/sparkfun/Arduino_Boards/master/IDE_Board_Manager/package_sparkfun_index.json

Más preferencias pueden ser editadas directamente en el fichero
C:\Users\Administrador\AppData\Local\Arduino15\preferences.txt
(editar sólo cuando Arduino no está corriendo)

Ok Cancelar

Gestor de URLs Adicionales de Tarjetas:

Introduzca URLs adicionales, una por cada fila

http://arduino.esp8266.com/stable/package_esp8266com_index.json
https://raw.githubusercontent.com/sparkfun/Arduino_Boards/master/IDE_Board_Manager/package_sparkfun_index.json

Clique para obtener una lista de las URLs de soporte para las tarjetas no oficiales

Ok Cancelar



POLITÉCNICA

Hardware para internet de las cosas (IoT)

Fuente: Arduino

Selección del hardware

Blink Arduino 1.8.12

Archivo Editor Programa Herramientas Ayuda

Nuevo Ctrl+N
Abrir... Ctrl+O
Abrir Reciente
Proyecto
Ejemplos
Cerrar Ctrl+W
Salvar Ctrl+S
Guardar Como... Ctrl+Mayús+S
Configurar Página Ctrl+Mayús+P
Imprimir Ctrl+P
Preferencias Ctrl+Coma
Salir Ctrl+Q

Preferencias

Ajustes Red

Localización de proyecto
C:\Users\Administrador\Documents\Arduino Explorar

Editor de idioma: System Default (requiere reiniciar Arduino)
Editor de Tamaño de Fuente: 12
Escala Interfaz: ☒ Automático 100% (requiere reiniciar Arduino)
Tema: Tema por defecto (requiere reiniciar Arduino)
Mostrar salida detallada mientras: ☐ Compilación ☐ Subir
Advertencias del compilador: Ninguno
☐ Mostrar números de línea ☐ Habilitar Plegado Código
☒ Verificar código después de subir ☐ Usar editor externo
☒ Comprobar actualizaciones al iniciar ☒ Guardar cuando se verifique o cargue
☐ Use accessibility features

Introduzca URLs adicionales, una por cada fila

http://arduino.esp8266.com/stable/package_esp8266com_index.json
https://raw.githubusercontent.com/sparkfun/Arduino_Boards/master/IDE_Board_Manager/package_sparkfun_index.json

Clique para obtener una lista de las URLs de soporte para las tarjetas no oficiales

Ok Cancelar

Subido

El Sketch usa 3938 bytes (13%) del
Las variables Globales usan 149 by

https://raw.githubusercontent.com/sparkfun/Arduino_Boards/master/IDE_Board_Manager/package_sparkfun_index.json



Selección del hardware

The screenshot shows the Arduino IDE interface. The top menu bar includes 'Archivo', 'Editar', 'Programa', 'Herramientas', and 'Ayuda'. The 'Gestor de tarjetas' (Board Manager) window is open, displaying a list of boards. The 'Tipo' (Type) dropdown is set to 'Todos' (All), and the search filter is 'sparkfun'. The list includes 'Tarjetas incluidas en este paquete' (Boards included in this package) and 'SparkFun AVR Boards' (by SparkFun Electronics, versión 1.1.13, INSTALLED). The 'SparkFun AVR Boards' section lists boards like RedBoard, MaKey MaKey, Pro Micro, Fio v3, Qduino Mini, Digital Sandbox, Mega Pro, RedBot, Serial 7-segment Display, ATmega128RFA1 Dev Board, LilyPad USB Plus, and SerLCD. Below this, the 'SparkFun SAMD Boards (dependency: Arduino SAMD Boards 1.8.1)' section lists boards like SparkFun SAMD21 Mini Breakout, SparkFun SAMD21 Dev Breakout, SparkFun 9DoF Razor IMU M0, LilyPad LilyMini, Sparkfun SAMD21 Pro RF, Sparkfun RedBoard Turbo, Sparkfun SAMD51 Thing Plus, and Sparkfun Qwiic Micro. The 'Subido' (Uploaded) status is shown at the bottom left, indicating the sketch is ready. The bottom right shows 'Ok' and 'Cancelar' buttons.

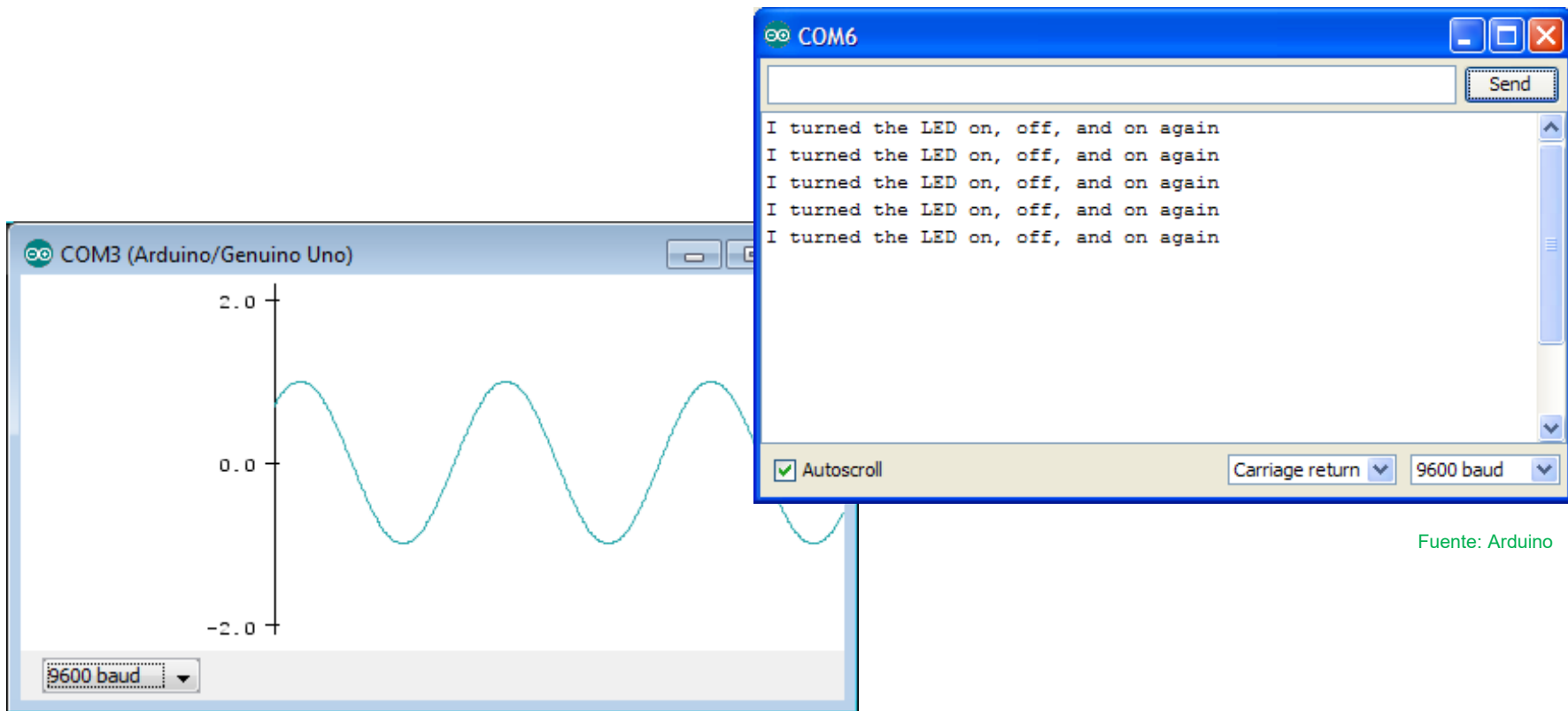
El Sketch usa 3938 bytes (13%) del
Las variables Globales usan 149 by



Opciones de depuración

- ❑ Dos ventanas de ayuda a la depuración:
 - Una consola donde se deben enviar mensajes.

`Serial.println("I turned the LED on, off, and on again");`



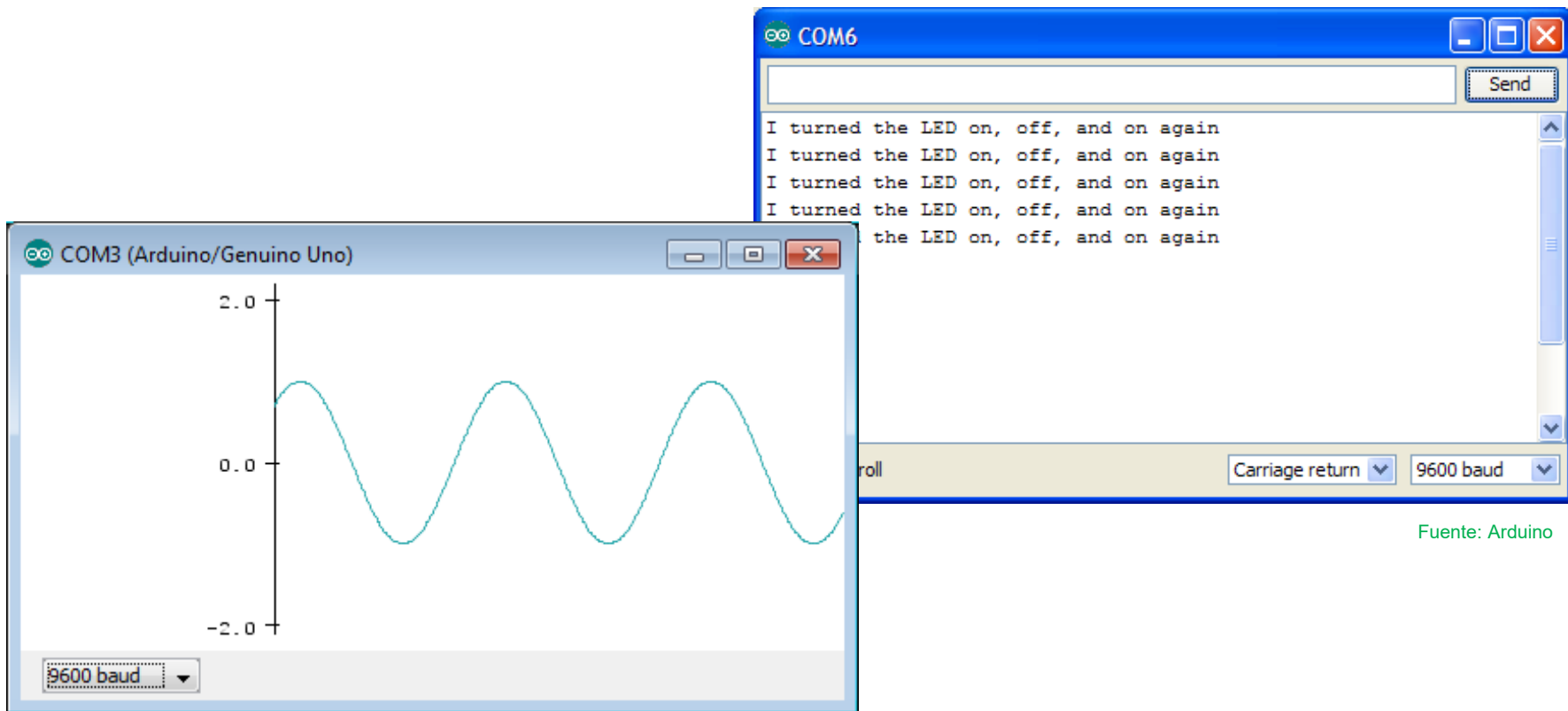
Fuente: Arduino



Opciones de depuración

- ❑ Dos ventanas de ayuda a la depuración:
 - Una consola que funciona a modo de plotter.

`Serial.println(value);`

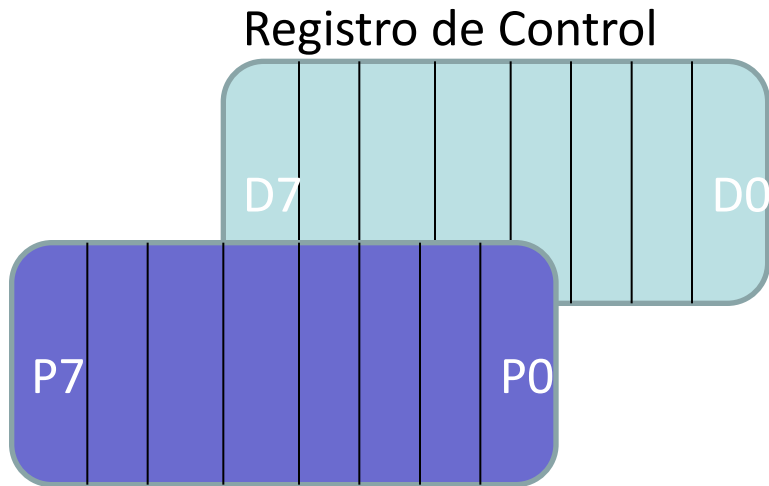


Fuente: Arduino



Elementos de entrada/salida

- Independientemente de si el periférico está integrado, está mapeado en memoria o conectado al microprocesador mediante algún estándar de comunicaciones, la entrada/salida es gestionada conceptualmente mediante dos registros.



Registro de Control: Se utiliza para determinar la dirección del puerto (Entrada o Salida).

Puerto: Se corresponde con el pin del C.I. De forma que el valor que se escriba en un bit del puerto será el que se medirá en el correspondiente pin. Si se lee el bit de un puerto, se leerá el valor lógico que existe en el pin.

Elementos de entrada/salida

	VIA 6522	C8051F020	ATMega128	LPC1764
Registro de Control	Data Direction Register (DDR)	Port Output Mode Register (PMD0U)	Data Direction Register (DDR)	Fast GPIO Port Direction (FIODIR)
Puerto	Input Register/Output Register (IRA/ORR)	Data Register (PORT)	Data Register (PORT)	Fast Port Pin value (FIOPIN)

En cada arquitectura reciben nombres distintos pero el concepto es el mismo:

1. Primero se selecciona el comportamiento del Pin/Puerto.

2. Se accede al Pin/Puerto en lectura y/o escritura

LED_BUILTIN= en Lillypad USB+ se corresponde con el LED RGB_Verde

```
void setup() {  
  
    pinMode(LED_BUILTIN, OUTPUT);  
}
```

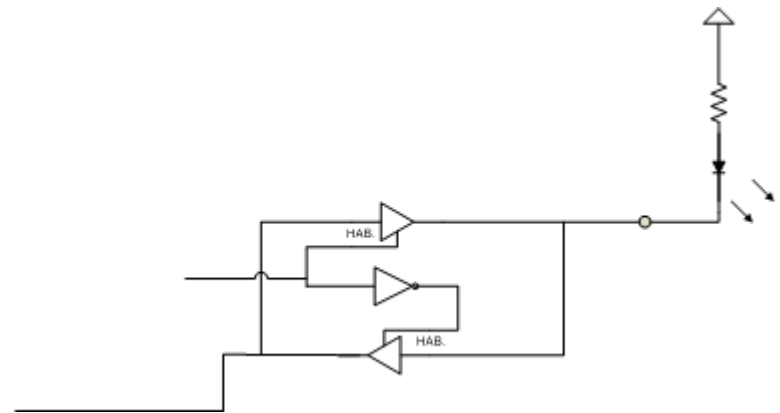
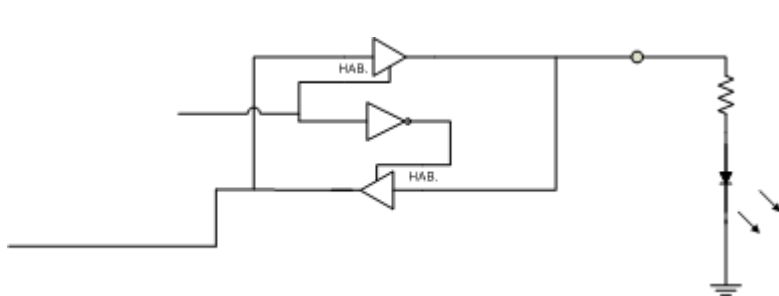
Específicamente cada arquitectura amplía la funcionalidad con diferentes prestaciones :

1. Salidas en colector abierto.
2. Líneas de handshake.
3. Etc.

```
void loop() {  
    digitalWrite(LED_BUILTIN, HIGH);  
    delay(1000);  
    digitalWrite(LED_BUILTIN, LOW);  
    delay(1000);  
}
```



Conexionado E/S



- La corriente que drena o que aporta un pin está limitada. Suele ser del orden de 20 mA, hay que verificarlo en el datasheet.

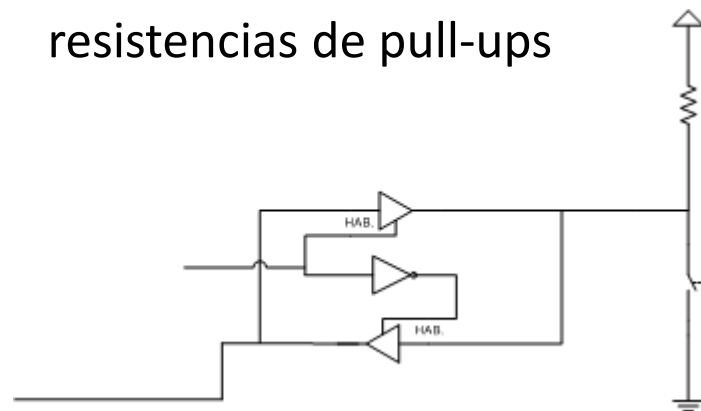
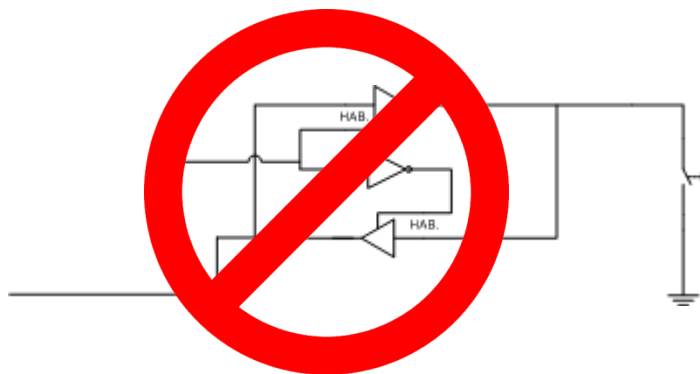
Conexionado E/S

Gestión entradas.

El dispositivo más común para realizar la entrada a un sistema microprocesador es un **pulsador/interruptor**. Ya sea de forma independiente o bien formando parte de un conjunto (**teclado**).

¿Cómo se realiza la conexión?

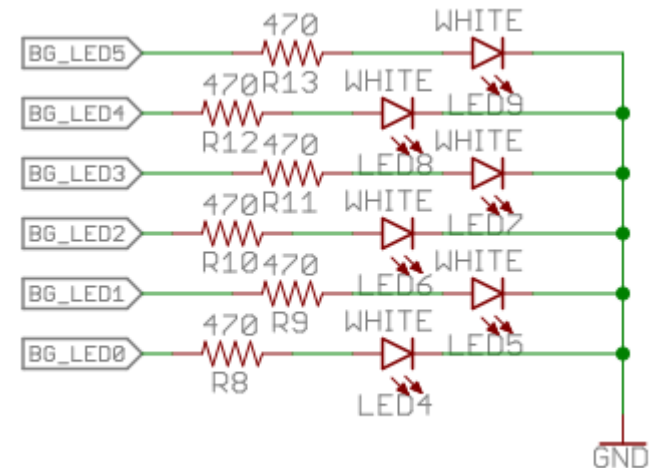
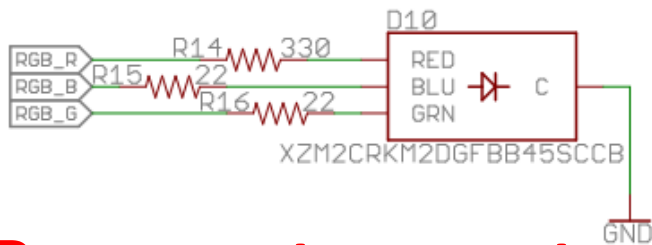
Necesario utilizar resistencias de pull-ups



LEDs en Lilypad

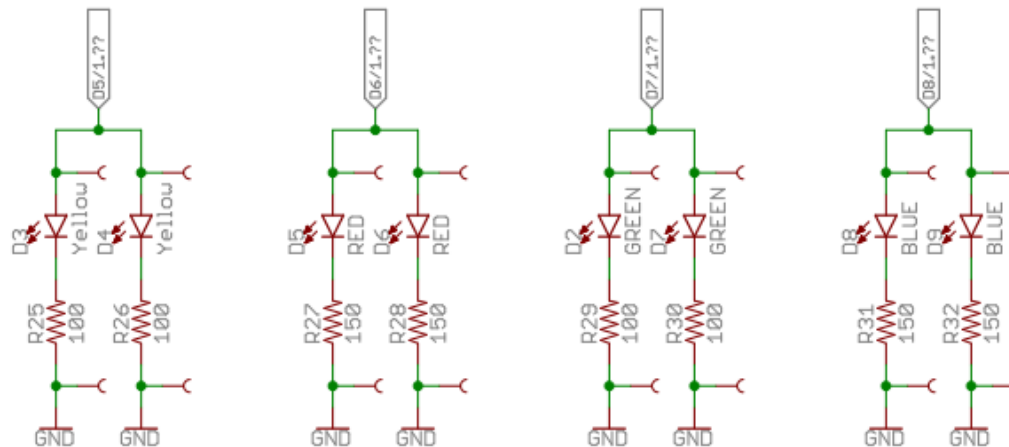
❑ Montados en Placa principal

- Barra de Leds blancos.
- Led RGB.



❑ Para montar en e-textil

- Cuatro parejas de leds (Amarillo, Rojo, Verde y Azul).

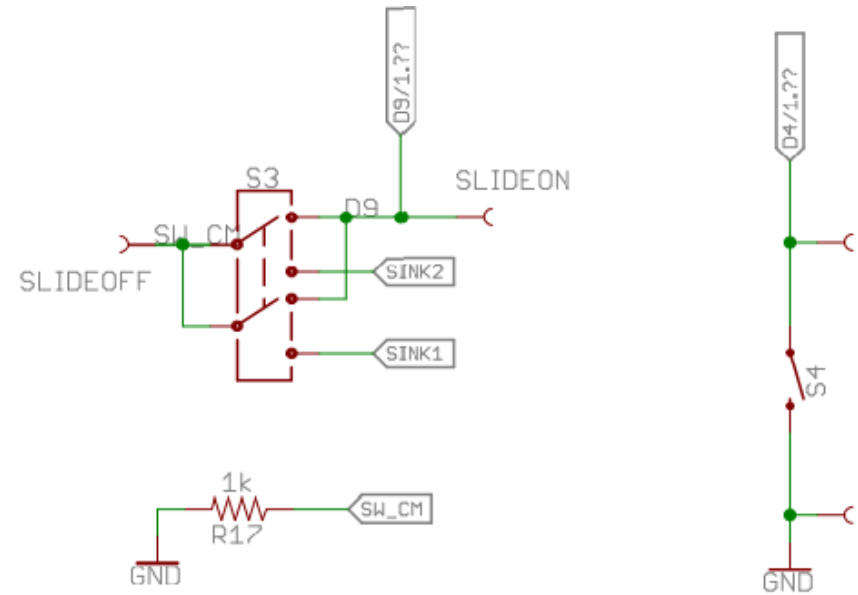


Fuente: Sparkfun Electronics



Interruptores en Lilypad

- ❑ Se debe configurar el pin como entrada.
- ❑ Hay que utilizar resistencias de pull-up o pull-down para obtener un nivel válido.
- ❑ Se pueden conectar externamente o bien utilizar las que proporciona el uP.



Fuente: Sparkfun Electronics

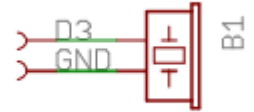
```
pinMode(pin, INPUT);           // set pin to input
digitalWrite(pin, HIGH);        // turn on pullup resistors
```

Nuevas versiones Arduino permiten manejar los pull-ups internos del uP.
INPUT_PULLUP



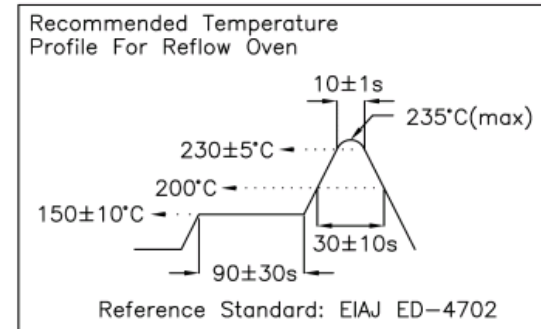
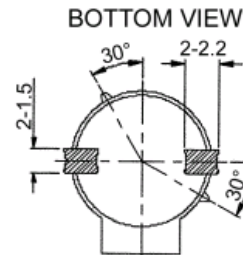
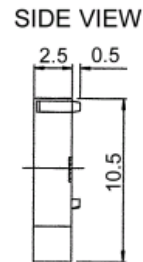
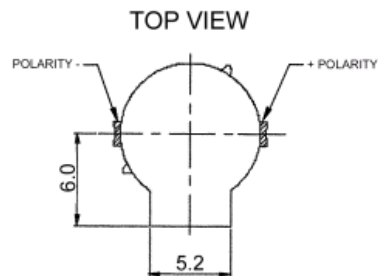
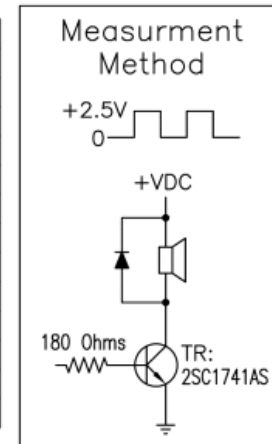
Otros periféricos digitales (salida)

- ❑ Configurar los pines como salidas.
- ❑ Prestar especial atención a la polaridad en las conexiones.

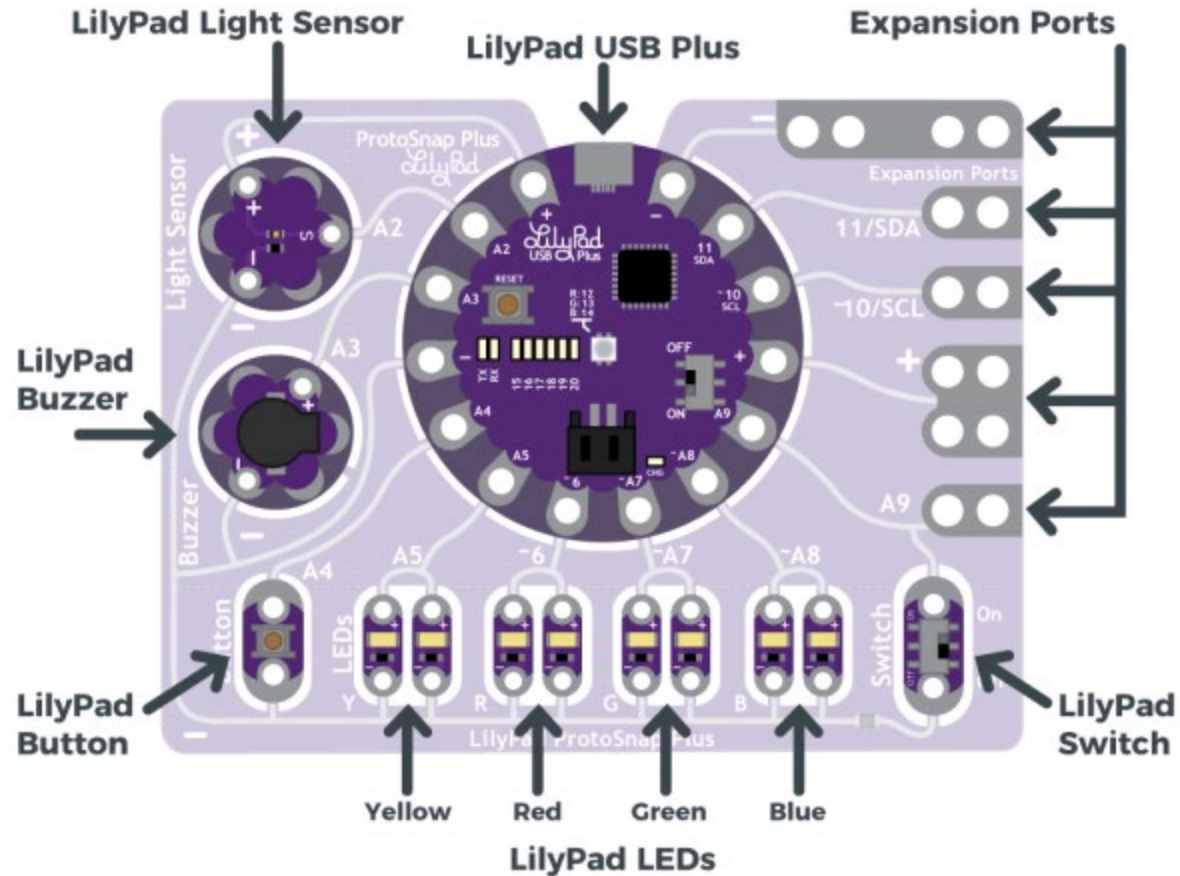


Test Condition: Temp=+25°C±2°C Relative Humidity=65±5% Air Pressure:860–1060mbar

No.	Item	Specification
1	Rated Voltage	3.6 Vo-p
2	Operating Volt.	2.5~4.5 Vo-p
3	Mean Current	Max. 110 mA Applying Rated Voltage, 2730Hz, Square Wave, 1/2 Duty
4	Coil Resistance	16.0±2.4 Ohms
5	Coil Impedance	31.0 Ohms
6	Sound Output	Min. 85 dBA (90 Typ.) Distance at 10cm(A-Weight). Applying rated voltage 2730Hz square wave, 1/2 duty.
7	Rated Frequency	2730Hz
8	Operating Temp.	-30~+70°C
9	Storage Temp.	-40~+85°C
10	Dimension	ø9.0 x H2.4 mm
11	Weight	0.6 grams
12	Material	PPS
13	Terminal	SMD Type



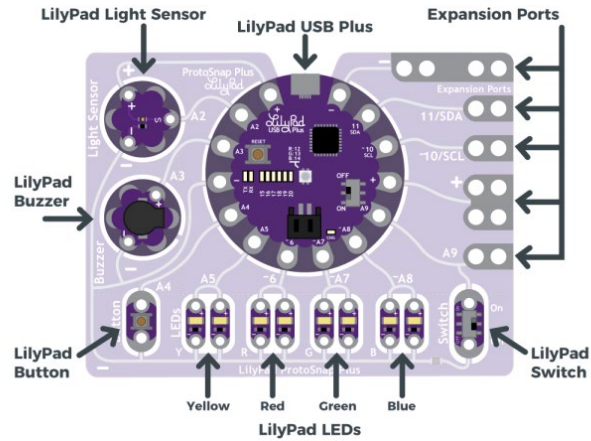
Periféricos Layout



Fuente: Sparkfun Electronics



Periféricos Layout

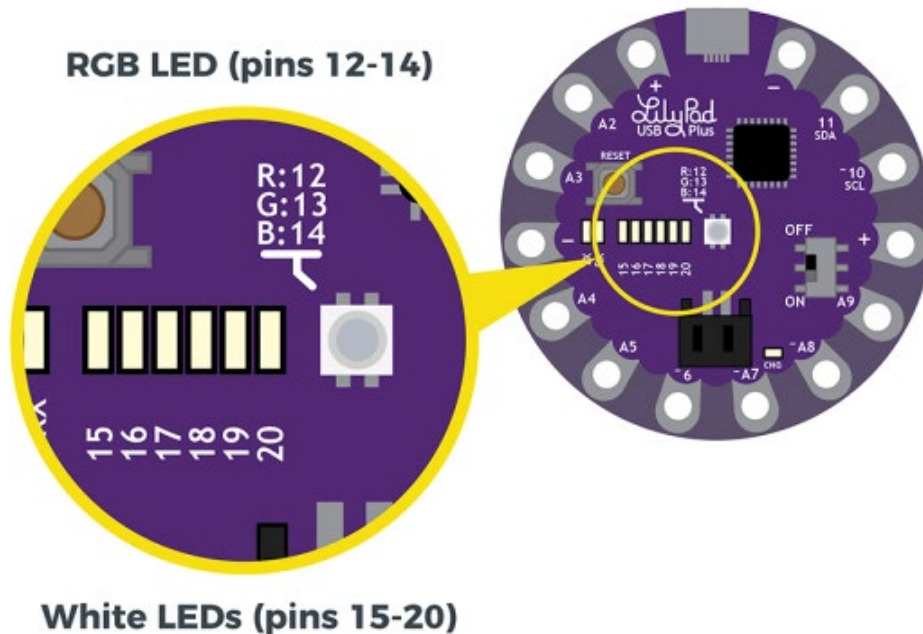


Conexionado exterior	Digital	Analógico
I/O	2	A2
I/O	3	A3
I/O	4	A4
I/O	5	A5
I/O	~6	
I/O	~7	A7
I/O	~8	A8
I/O	9	A9
SCL	~10	
SDA	11	

Fuente: Sparkfun Electronics



Periféricos Layout



Conexionado interior	Digital
RGB-Led Rojo	~12
RGB-Led Verde	~13
RGB-Led Azul	~14
Barra-Led 0	15
Barra-Led 1	16
Barra-Led 2	17
Barra-Led 3	18
Barra-Led 4	19
Barra-Led 5	20

Fuente: Sparkfun Electronics



Funciones para la gestión de E/S digitales

Functions

Digital I/O

- `pinMode()`
- `digitalWrite()`
- `digitalRead()`

`pinMode()`

Description

Configures the specified pin to behave either as an input or an output. See the description of [digital pins](#) for details on the functionality of the pins.

As of Arduino 1.0.1, it is possible to enable the internal pullup resistors with the mode `INPUT_PULLUP`. Additionally, the `INPUT` mode explicitly disables the internal pullups.

Syntax

```
pinMode(pin, mode)
```

Parameters

pin: the number of the pin whose mode you wish to set

mode: `INPUT`, `OUTPUT`, or `INPUT_PULLUP`. (see the [digital pins](#) page for a more complete description of the functionality.)

Fuente: Arduino



Funciones para la gestión de E/S digitales

Functions

Digital I/O

- `pinMode()`
- `digitalWrite()`
- `digitalRead()`

`digitalWrite()`

Description

Write a `HIGH` or a `LOW` value to a digital pin.

If the pin has been configured as an OUTPUT with `pinMode()`, its voltage will be set to the corresponding value: 5V (or 3.3V on 3.3V boards) for HIGH, 0V (ground) for LOW.

If the pin is configured as an INPUT, `digitalWrite()` will enable (HIGH) or disable (LOW) the internal pullup on the input pin. It is recommended to set the `pinMode()` to `INPUT_PULLUP` to enable the internal pull-up resistor. See the [digital pins tutorial](#) for more information.

Fuente: Arduino



Funciones para la gestión de E/S digitales

Functions

Digital I/O

- `pinMode()`
- `digitalWrite()`
- `digitalRead()`

digitalRead()

Description

Reads the value from a specified digital pin, either HIGH or LOW.

Syntax

`digitalRead(pin)`

Parameters

pin: the number of the digital pin you want to read (*int*)

Returns

HIGH or LOW

Fuente: Arduino



Conversión Analógico/Digital y Digital-Analógico

El mundo que nos rodea es analógico: temperatura, humedad, sonido, etc. Sin embargo hoy en día la mayoría de los equipos trabajan con datos digitales: Reproductores CD, Radio DAB, etc.

Realizar el procesamiento de forma digital presenta numerosas ventajas: Sistemas más inmunes al ruido, sistemas con capacidad de reprogramación, más económicos, etc.

Es necesario disponer de dispositivos que permitan convertir las magnitudes físicas en eléctricas. Esta función la realizan los sensores/transductores. Ej: RTD, LVDT, Micrófonos, Altavoz, etc.

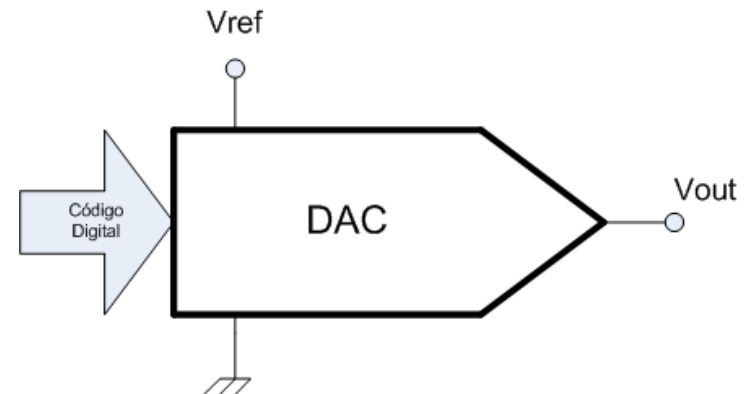
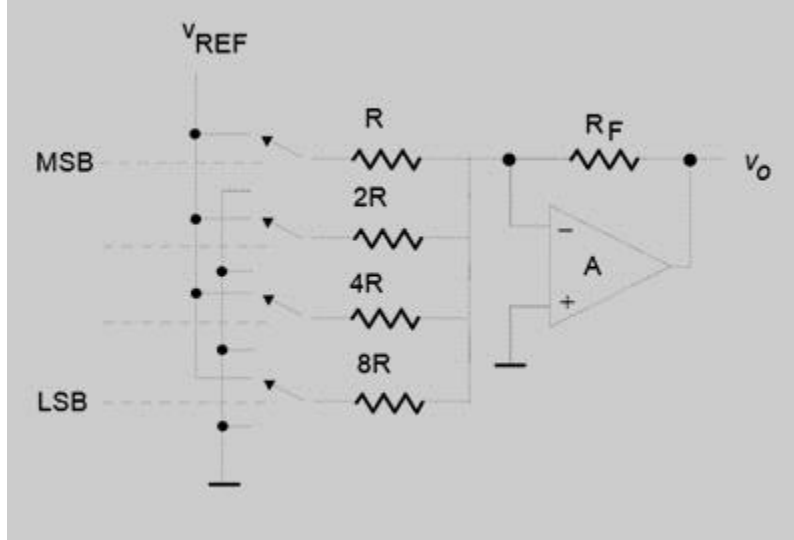
Para poder procesar las señales que proporcionan los sensores es necesario disponer de circuitos electrónicos que realicen la conversión Analógico-Digital y de Digital-Analógico.



Conversión Digital/Analógico

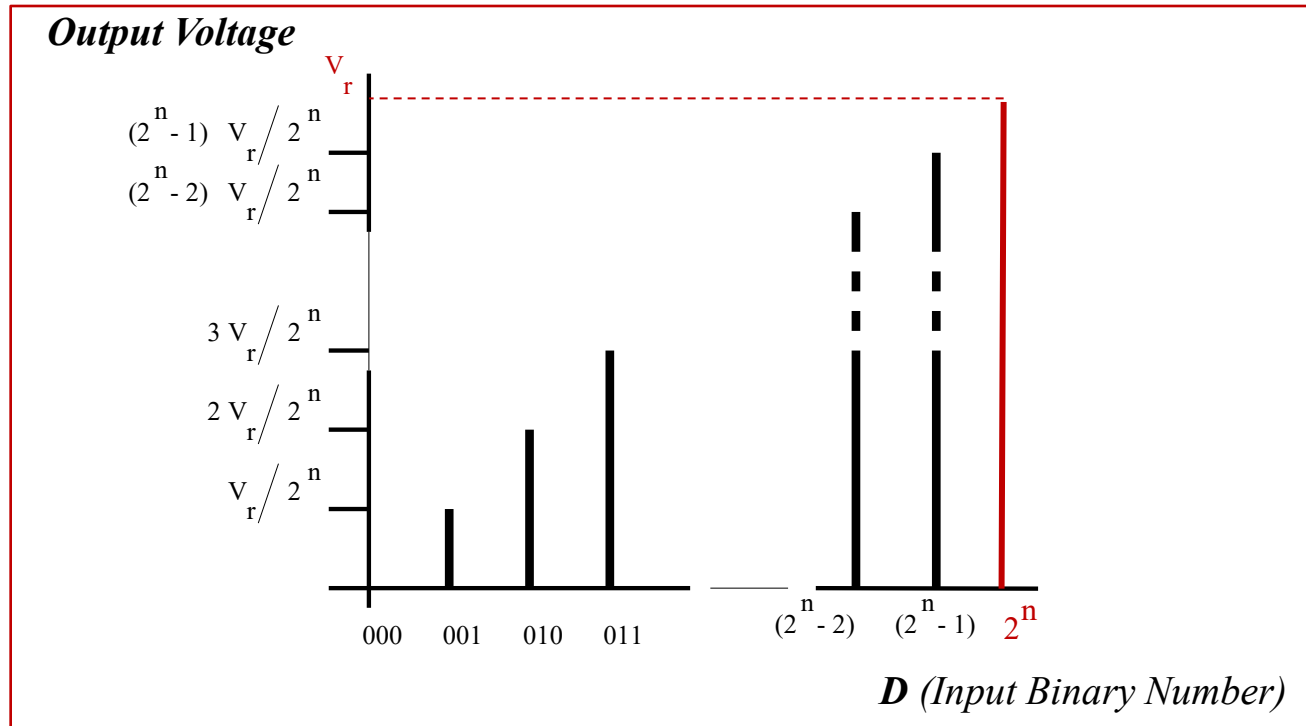
Un convertidor analógico digital es un circuito electrónico que convierte códigos digitales en una tensión analógica proporcional a dicho código.

Los convertidores digitales-analógicos son circuitos evolucionados de un diagrama **básico** como el indicado a continuación.



Conversión Digital/Analógico

La función de transferencia básica de un convertidor digital-analógico es la mostrada a continuación.



La tensión de salida es:

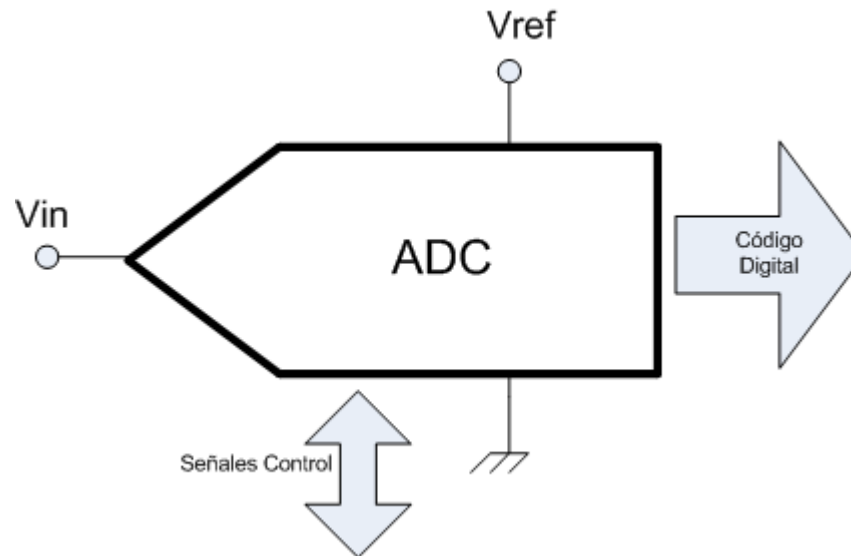
$$V_o = CD \frac{V_{ref}}{2^n}$$

Hardware para internet de las cosas (IoT)

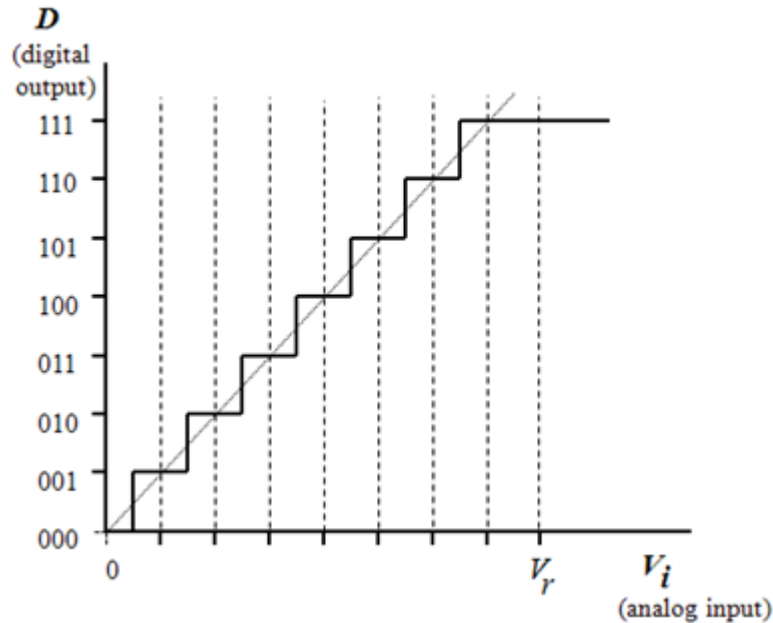


Conversión Analógico-Digital

Un convertidor analógico-digital es un dispositivo electrónico que entrega un código digital proporcional a su tensión de entrada.



Función de transferencia de un convertidor Analógico-digital



$$D = \frac{V_i}{V_r} \times 2^n$$

$$LSB = \frac{V_{ref}}{2^n}$$

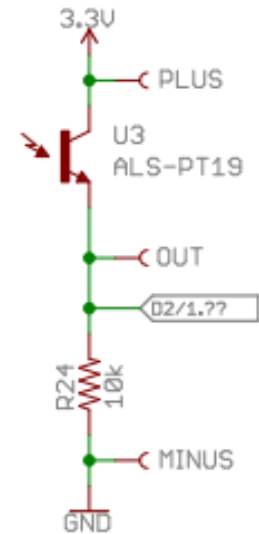
Rango de entrada es la diferencia entre el valor máximo de la tensión de entrada y el valor mínimo.

Resolución: Mínima variación en la tensión de entrada que produce un cambio de código digital a la salida.

Entradas Analógicas

- ❑ Adecuar la salida del sensor al rango de entrada del convertidor.
- ❑ Fijar la tensión de referencia que utiliza el convertidor.
- ❑ Determinar el LSB y obtener la función de transferencia.

Light Sensor



Fuente: Sparkfun Electronics



Funciones para la gestión de entradas analógicas

Functions

Analog I/O

- `analogReference()`
- `analogRead()`

analogReference()

Description

Configures the reference voltage used for analog input (i.e. the value used as the top of the input range). The options are:

- DEFAULT: the default analog reference of 5 volts (on 5V Arduino boards) or 3.3 volts (on 3.3V Arduino boards)
- INTERNAL: an built-in reference, equal to 1.1 volts on the ATmega168 or ATmega328 and 2.56 volts on the ATmega8 (*not available on the Arduino Mega*)
- INTERNAL1V1: a built-in 1.1V reference (*Arduino Mega only*)
- INTERNAL2V56: a built-in 2.56V reference (*Arduino Mega only*)
- EXTERNAL: the voltage applied to the AREF pin (**0 to 5V only**) is used as the reference.

Syntax

`analogReference(type)`

Parameters

type: which type of reference to use (DEFAULT, INTERNAL, INTERNAL1V1, INTERNAL2V56, or EXTERNAL).

Returns

None.

Note

After changing the analog reference, the first few readings from `analogRead()` may not be accurate.

Fuente: Arduino



Funciones para la gestión de entradas analógicas

Functions

Analog I/O

- `analogReference()`
- `analogRead()`

analogRead()

Description

Reads the value from the specified analog pin. The Arduino board contains a 6 channel (8 channels on the Mini and Nano, 16 on the Mega), 10-bit analog to digital converter. This means that it will map input voltages between 0 and 5 volts into integer values between 0 and 1023. This yields a resolution between readings of: 5 volts / 1024 units or, .0049 volts (4.9 mV) per unit. The input range and resolution can be changed using `analogReference()`.

It takes about 100 microseconds (0.0001 s) to read an analog input, so the maximum reading rate is about 10,000 times a second.

Syntax

```
analogRead(pin)
```

Parameters

pin: the number of the analog input pin to read from (0 to 5 on most boards, 0 to 7 on the Mini and Nano, 0 to 15 on the Mega)

Returns

int (0 to 1023)

Fuente: Arduino



Salidas analógicas

- Utilizan la técnica de la PWM.

